

Received 11 November 2024, accepted 4 December 2024, date of publication 9 December 2024,
date of current version 18 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3514215

RESEARCH ARTICLE

HENCE: Hardware End-to-End Neural Conditional Entropy Encoder for Lossless 3D Medical Image Compression

JIETAO CHEN¹, QIANHAO CHEN¹, HUAN ZHANG², WEIJIE CHEN¹, WEI LUO¹,
AND FENG YU¹, (Member, IEEE)

¹College of Biomedical Engineering & Instrument Science, Zhejiang University, Zhejiang 310063, China

²College of Communication Engineering, Hangzhou Dianzi University, Hangzhou 310018, China

Corresponding author: Feng Yu (osfengyu@zju.edu.cn)

ABSTRACT Recently, learning-based lossless compression methods for volumetric medical images have attracted much attention. They can achieve higher compression ratios than traditional methods, albeit at the cost of slower compression speed. Although using field programmable gate arrays (FPGAs) is feasible to mitigate this disadvantage, existing FPGA-based compression frameworks still need CPU for co-processing. In this work, we propose a hardware end-to-end neural conditional entropy encoder (HENCE), for losslessly compressing 3D medical images with the balanced compression ratio and speed. To achieve this, we first introduce a context-based entropy model to reduce data redundancy within inter-slice and intra-slice features, using an efficient combination of auto-regressive and recurrent neural networks. Then, we design a hardware-friendly arithmetic coding module to collaborate with our learning-based entropy model. To obtain the cumulative distribution function of the discrete logistic distribution, we further introduce a high-precision Sigmoid approximation algorithm, using the Newton-Raphson method. Finally, we design a dataflow mechanism for the entropy model and the coding module, achieving a fully pipelined compression system. Extensive experimental results show that our method outperforms traditional image/video codecs like FLIF, JPEX-XL, and HEVC on several volumetric medical image datasets. And our method obtains faster encoding speed than existing learning-based medical image compression frameworks.

INDEX TERMS Neural networks, volumetric image compression, entropy coding, FPGA.

I. INTRODUCTION

Extensive utilization of digital medical images, including CT and MRI, for clinical auxiliary diagnosis is well-recognized. As digital imaging technology continues to advance and medical devices achieve higher resolutions, the size and bit depth of digital medical images are progressively expanding. This trend presents challenges in the storage of digital medical images. Furthermore, the potential for loss and distortion in these images may introduce discrepancies in the diagnoses made by medical professionals. Therefore, the

exploration of 3D medical image algorithms centered on lossless compression holds paramount significance.

Many medical imaging systems adhere to the guidelines established by the Digital Imaging and Communications in Medicine (DICOM) organization. These guidelines specify the use of major image coding standards, like PNG [1], JP2K [2], JP3D [3]. Meanwhile, next-generation image compression algorithms with good compression ratios, like FLIF [4] and JPEG-XL [5], have emerged in recent years. The traditional video coding standards such as H.264/AVC [6], H.265/HEVC [7], FFV1 [8] and VVC [9] are also candidates for volumetric image compression algorithms. Although these traditional codecs are still prevalent in real scenarios, they all rely on hand-crafted modules, which cannot be

The associate editor coordinating the review of this manuscript and approving it for publication was Ines Domingues¹.

optimized based on massive training data, resulting in relatively poor compression ratios [10].

Thanks to significant advancements in deep learning, various deep neural network (DNN) based learning frameworks have rapidly emerged in the research field of image compression [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], achieving outstanding compression performance when compared to traditional methods. For example, auto-regressive (AR) algorithms like PixelCNN [19], PixelCNN++ [20], and DistAug [30], predict each symbol from the previous context. Auto-encoder (AE) methods are also one of the most common compressors. Mentzer et al. [22] introduces a hierarchical lossless image compression approach L3C. Bits-back coding schema [31] also improves compression ratio significantly [25], [26], [27]. Flow-based algorithms [28], [29], [32] apply learnable invertible transforms to make data easier to compress. As for volumetric medical images, based on L3C, Chen et al. [10] proposes an enhanced framework ICEC for volumetric medical images, which exploits both inter-slice and intra-slice feature information to further reduce data redundancy. Meanwhile, Xue et al. [21] introduces a 3D trained affine wavelet-like transform method called aiWave, which can be adaptive to different data and be feasible to handle various local contexts. Based on aiWave, Xue et al. [23] also proposes a video-based medical image compression method DBVC, which includes motion estimation and motion compensation. However, these methods suffer from drawbacks such as slow inference speed, large model size, and fixed resolution constraint, which may hinder the practical application of learning-based compression methods in real-world scenarios.

In recent years, regarding the growing demand for fast real-time applications, there are many advanced deep learning models deployed on field programmable gate arrays (FPGAs) and application specific integrated circuits (ASICs) [34], [35], [36]. For image compression implementations, Jia et al. [34] proposes an FPGA-accelerated lossy video coding system, which utilizes a local-content adaptive scheme of block partition for a more accurate entropy model. In addition, our previous work SR-LVC [24] proposes a stream-based lossless volumetric image compression framework. However, the limitation of existing hardware-accelerated image compression methods is that they still need software co-processing for the coding process. When the CPU encoding speed is slower than the network inference speed of the FPGA, it will limit the overall encoding performance.

This paper aims to propose a fully hardware end-to-end 3D medical image encoder, that can achieve low latency and fast streaming compression without CPU co-processing. However, there are still several challenges to obtaining this goal.

For instance, most existing neural methods for image compression pay more attention to the enhanced network design but neglect accelerating the entropy coding (e.g.

Algorithm Coding (AC) [37], Asymmetric Numeral System (ANS) [38]). Take L3C [22] and idf++ [32] as examples, at the beginning of the encoding process, they predict the conditional probability distribution of symbols. After that, they calculate the Cumulative Distribution Function (CDF) table for each symbol. Finally, they input these CDF tables into the entropy coding module to get the compressed data. However, when handling large bit-depth medical images, such as 16-bit CT/MRI images, it will impose a significant computational overhead, as computing the CDF table for one pixel requires 2^{16} iterations. One solution is to calculate the frequency range of the symbol that needs compressing through the cumulative distribution function directly.

Moreover, starting from PixelCNN++ [20], many learning based algorithms typically assume that the data follows the logistic distribution. However, the computation of CDF (i.e. Sigmoid function) for such distribution poses significant challenges for hardware implementation. Most existing Sigmoid approximation methods like [39], [40], [41], and [42] are primarily designed for use as activation functions in neural networks and may not be suitable for entropy coding. For example, the hard-Sigmoid function may have a limited impact on the final performance of some tasks like classification [43] when used as an activation function, but may significantly affect the compression ratio in entropy coding. Moreover, some methods like [42] do not satisfy the mathematical property of monotonicity, a basic requirement for the CDF. Last but not least, some methods [39], [42] restrict the input range into $(-8, 8)$, which is sufficient for an activation function but may fail to meet the accuracy requirement for entropy coding.

Finally, existing FPGA-accelerated DNN models [36], [44], [45] often exhibit structural similarities between layers, leading to the adoption of a layer-task dataflow. However, for entropy encoding like arithmetic coding, symbols are processed sequentially as in a bit-task dataflow. This kind of dataflow mismatch may lead to performance loss. One solution is to design a unified dataflow paradigm and allocate rational logic resources for both the entropy network model and the coding module.

To address these problems, this paper proposes a fully hardware end-to-end neural conditional entropy encoder (HENCE) for lossless 3D medical image compression, which is an upgraded version of our previous work SR-LVC [24]. Our framework consists of two components: a Probability Distribution Estimator (PDE) module and a Mixed Logistic Arithmetic Coding (MLAC) module. On the one hand, the PDE module plays a role in modeling the probability distributions of individual symbols. Following SR-LVC, we incorporate an auxiliary feature channel to transmit the prior context information of previous slices, further reducing intra-slice and inter-slice redundancy. Compared to SR-LVC, as shown in Table 1, HENCE has been optimized for lower computational complexity by replacing the Gate Fusion module with a 16-channel *tanh* function to reduce the number

TABLE 1. Comparison of SR-LVC and HENCE.

	Gate Module	Gate Number	Entropy Model	FPGA Support	
				Network Inference	Bits Encoding
SR-LVC [24]	48-channel GRU [33]	two	1-layer single logistic distribution	✓	×
Ours	16-channel <i>tanh</i>	one	2-layer mixture of logistic distributions	✓	✓

of network parameters. Moreover, we have redesigned the architecture by using the same feature-extracting modules such as the Masked Convolution Layer and the DSC Layer to estimate pixel-level distribution probabilities and update hidden auxiliary features simultaneously, leading to a decrease in the number of Gate Fusion modules. We use a mixture of logistic distributions in the PDE as compensation for accuracy. On the other hand, our MLAC module aims to encode symbols into binary bitstreams, based on the estimated distribution parameters. To reduce the computational complexity of the CDF table, especially for high-frequency images, we use the cumulative function to calculate the frequency range directly during encoding. However, for decoding, the CDF table is still computed as usual. Then we also present a specialized Sigmoid approximation algorithm for arithmetic coding, through the two-step Newton-Raphson method with high precision. Finally, we design a unified pipeline dataflow for the entropy model PDE and the coding module MLAC, where input symbols can be sequentially encoded in a pixel-level paradigm.

Our contributions can be summarized as follows:

- To the best of our knowledge, it is the first hardware end-to-end learning-based encoder for lossless 3D medical image compression, which can achieve low latency and fast streaming compression without CPU co-processing.
- We propose a simple baseline of context entropy model (with only 1513 weighting parameters) called PDE and a hardware-friendly coding module called MLAC, the collaboration of which efficiently improves the image coding speed of learning-based methods, only 5 milliseconds for one 256×256 image slice.
- We also propose a high-precision approximation method for the logistic cumulative distribution function through the Newton-Raphson method, enabling hardware employment without sacrificing compression ratio.

This paper is organized as follows. Section II describes the detailed principles of our framework. Section III shows the hardware architecture for FPGA implementation. Section IV presents the experimental results and analysis. Section V concludes the entire paper.

II. METHOD

This section introduces our proposed lossless 3D image compression framework HENCE in detail. First, we give an overview of the entire framework. Second, we introduce the entropy model of the conditional probability distribution estimator (PDE). Finally, we discuss the strategies of the mixed logistic arithmetic coding (MLAC) module.

A. OVERVIEW

Characteristics of volumetric medical images, such as CT and MRI images, differ from natural images. First, unlike natural images that use RGB or YUV formats, medical images are typically single-channel grayscale images, with a bit depth of up to 16. Additionally, 3D medical images contain a significant amount of inter-frame redundancy between slices. Although existing video compression algorithms can also compress volumetric data, most use the reference frame and the current frame to extract inter-slice information through motion vector estimation and compensation. Given that 3D medical images do not have a clear concept of background and moving objects between frames, video compression methods are counter-intuitive for these images [21]. In our approach, we propose an efficient framework that exploits inter-slice and intra-slice features simultaneously and uses hidden auxiliary features to cache information from all of the previous frames.

Fig. 1 shows our proposed lossless 3D image compression framework HENCE, which mainly includes two modules: the learning conditional probability distribution estimator (PDE) module and the mixed logistic arithmetic coding (MLAC) module. The overall process of compressing one slice of the medical image volume is illustrated as follows.

Let $V = (X_1, X_2, \dots, X_T)$ represent the volumetric data (e.g., the MRI or CT data) with T slices, while X_s represents the s^{th} slice of this volumetric data. According to the theory of auto-regressive (AR) methods, predicting the current symbol based on the previous symbol context is possible. Moreover, we employ a neural network to model the probability distribution of pixels (see more details in Section II-B). In this way, we can obtain the conditional probability distribution $\hat{p}_s$ for the current pixel x_i and the joint probability distribution of slice X_s as shown in (1), where $\cdot$ represents the previous symbol context:

$$P(X_s) = \prod_i \hat{p}_s(x_i | \cdot). \quad (1)$$

In this paper, we assume that the data distribution of symbols follows a mixture of logistic distributions. To achieve better compression ratios, we can minimize the cross-entropy between the actual data distribution and the estimated entropy model, which is similar to the loss function:

$$\begin{aligned} E(X, P) &= \mathbb{E}_{X \sim P}(-\log(P(X_s))) \\ &= \mathbb{E}_{X \sim P}(-\log(\prod_i \sum_{k=1}^K \pi_k \hat{p}(x_i | \cdot))), \end{aligned} \quad (2)$$

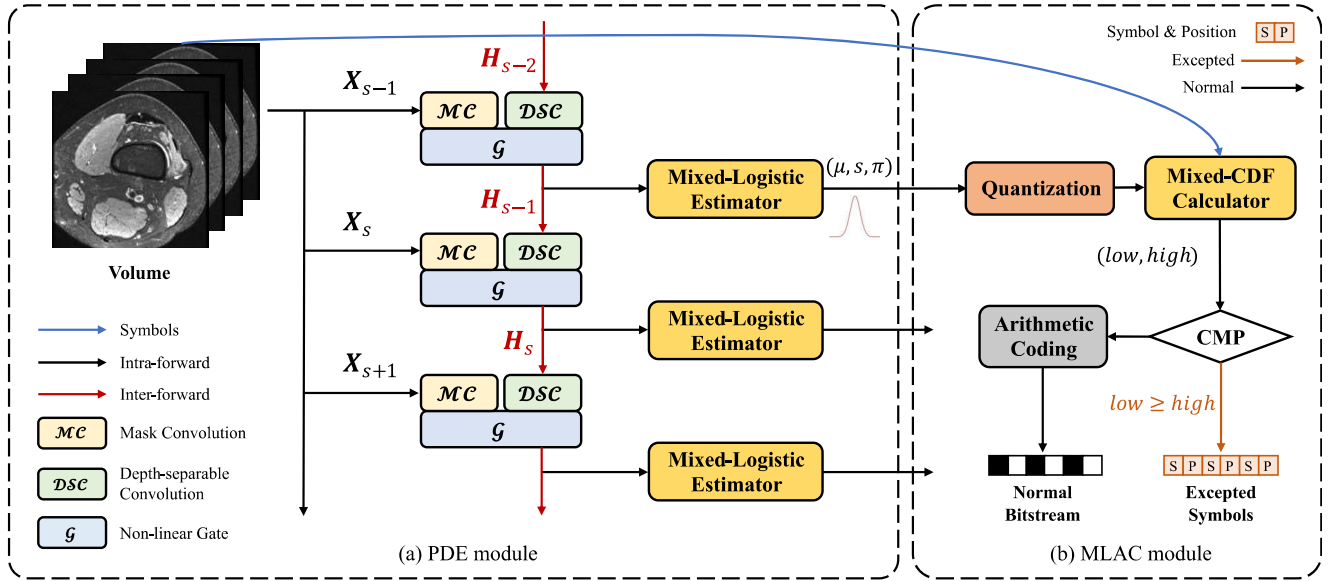


FIGURE 1. The details of our proposed HENCE framework, which includes (a) the context entropy model PDE and (b) the coding module MLAC. The PDE module first uses a masked convolution layer to extract current-slice features in the intra-forward way as X_s , a depthwise separable convolution (DSC) layer to extract prior-slice features in the inter-forward way as H_{s-1} , and a non-linear gate to update H_s by fusing these two features. Then, the mixed-logistic estimator module in the PDE outputs parameter tuples of each distribution as (μ, s, π) , representing mean, scale, and probability separately, which will be utilized sequentially by the MLAC module. In detail, with these parameters after quantization and the original symbol values, the MLAC module calculates frequency range pairs of the current pixel as $(low, high)$ through the cumulative distribution function (CDF). If $low < high$, as in the normal condition, the MLAC will use the arithmetic coding to output compressed bitstreams. Otherwise, it will output the excepted symbol frames, which include exceptional symbol values and corresponding positions.

where K represents the total mixture number of logistic distributions, and π_k represents the corresponding weight.

Thus, we can obtain the logistic distribution parameters of (π, μ, s) for each pixel from the PDE module. Based on these parameter tuples, we further compute the CDF results of each pixel, where σ means the Sigmoid function:

$$F(x|\pi, \mu, s) = \sum_{k=1}^K \pi_k \sigma((x - \mu_k)/s_k). \quad (3)$$

To implement this method to the hardware platform, we also present a high-precision Sigmoid approximation approach based on the Newton-Raphson Method, and more details will be introduced in Section II-D. Finally, considering the causality of the AR method, we propose an adaptive arithmetic coding method for the mixture of logistic distributions, aiming to encode symbols into binary bitstreams.

B. PDE MODULE

Motivated by previous works ICEC [10] and SR-LVC [24], we exploit both inter-slice and intra-slice features to further reduce the data redundancy, the breakdown of which is shown in Fig. 1(a). We integrate the inter-forward way H and the intra-forward way X through the masked convolution function $\mathcal{MC}$, the depthwise separable convolution function $\mathcal{DSC}$, and the non-linear gate mechanism $\mathcal{G}$. The whole process can be expressed as follows:

$$H_s = \mathcal{G}(\mathcal{MC}(X_s), \mathcal{DSC}(H_{s-1})), \quad (4)$$

where the H_s means the auxiliary feature of s^{th} slice, which contains feature representation of both the current slice and previous context.

Specifically, different from the gate mechanism proposed in ICEC which contains heavy weighting parameters, or that proposed in SR-LVC which utilizes a GRU-based multi-channel gate mechanism, we introduce an extremely simplified and resource-efficient alternative as the gate $\mathcal{G}$. Considering the availability of hardware implementation, we directly use the Hardtanh [46] in $\mathcal{G}$:

$$\begin{aligned} \mathcal{G}(a, b) &= \text{Hardtanh}(a + b) \\ &= \begin{cases} 1, & a + b > 1 \\ -1, & a + b < -1 \\ a + b, & \text{otherwise} \end{cases} \end{aligned} \quad (5)$$

Finally, we use the mixed-logistic estimator to model the probability distributions of the current slice, which consists of two pointwise convolution (i.e. 1×1 convolution) layers. For each symbol that needs to be compressed, we then have the estimated logistic parameter tuples of (π, μ, s) , with the total number of K , which will be transferred as the input of the MLAC module later.

C. MLAC MODULE

To further speed up the hardware end-to-end image compression, we first introduce a hardware-friendly arithmetic coding module based on the mixture of logistic distributions, as shown in Fig. 1(b). This module mainly includes three

sub-modules: the quantization, the mixed-CDF calculator, and the coding sub-modules.

1) THE QUANTIZATION MODULE

is a kind of post-training processing, which serves two main purposes. One is to convert the continuous number of mean parameter μ into discrete values. Suppose the bit depth of input images is N , then we can quantize the mean parameter as follows, where round function $\mathcal{R}$ can be replaced as truncation function $\mathcal{T}$. And $\ll$ and $\gg$ represent the bit-shifting operations, which are commonly used in hardware description languages:

$$\begin{aligned}\mu &= \mathcal{R}(\mu \ll N) \gg N \\ &= \mathcal{T}((\mu \ll N) + 0.5) \gg N.\end{aligned}\quad (6)$$

Another purpose of the quantization operation is to calculate the scale parameter s from the output of the PDE module. Considering the numerical stability and computational convenience, neural networks usually output the logarithm of the scale parameter $\log(s)$. Although utilizing an exponential function to recover the scale parameter is easy for software, it still poses a challenge for hardware approximation. To address this issue, we introduce a two-step quantized lookup table approach to implement the exponential calculation of $\log(s)$:

$$\log(s) = \mathcal{T}(\log(s) \ll E) \gg E, \quad (7)$$

$$s = LUT(\log(s)), \quad (8)$$

where $\mathcal{T}$ represents the truncation operation, E represents the quantization stride of the logarithm of scale s . For hardware implementation, LUT represents an operation to restore the value of scale s by querying the lookup table. More experimental evidence shown in Section IV-D demonstrates this quantization-based method does not introduce a significant performance degradation on the final compression ratio.

2) THE MIXED-CDF CALCULATOR MODULE

serves to take the quantized output of the PDE module as input, compute the cumulative probability of the corresponding mixed-logistic distributions, and obtain symbol frequency range pairs (*low*, *high*). These pairs are then fed into the arithmetic coding module.

Reminding of the challenge we discussed earlier that many existing image compression methods [20], [22], [32] often compute the whole CDF table for coding one symbol. For example, to encode a pixel of N bit depth, they first calculate a CDF table array A with the size of 2^N , and then obtain the frequency range by looking up this array, where F represents the CDF introduced as (3):

$$A[i] = F(i), i = 1, 2, \dots, 2^N, \quad (9)$$

$$f_{low}(x) = A[x], \quad (10)$$

$$f_{high}(x) = A[x + 1]. \quad (11)$$

However, this method may pose a computational burden, especially when N is large for high-bit-depth images.

Although the CDF table is necessary for the decoding process since we need to find the decoded symbol within (*low*, *high*), we propose an efficient strategy to accelerate the encoding process. To Assume that the PDE model predicting the current symbol x is based on K mixed-logistic distributions, where the parameters of the k^{th} logistic distribution are denoted as μ_k and s_k . The frequency range for the current symbol x can be expressed by the following formula:

$$f_{low}(x|\pi, \mu, s) = \sum_{k=1}^K \pi_k \sigma(((x - 0.5) \gg N - \mu_k)/s_k), \quad (12)$$

$$f_{high}(x|\pi, \mu, s) = \sum_{k=1}^K \pi_k \sigma(((x + 0.5) \gg N - \mu_k)/s_k), \quad (13)$$

where σ represents the CDF of logistic distribution (i.e. Sigmoid). While considering hardware adaptability, we also propose a high-precision method to approximate the σ , more details are shown in Section II-D.

Moreover, the probability parameter π of each logistic distribution needs to satisfy a condition that the sum of all probability parameters of the current symbol should equal 1. To enforce this constraint, a common approach is to apply a Softmax operation. However, implementing Softmax on a hardware platform may also be challenging due to the exponential computation. As an alternative, we employ a simple yet effective two-step normalization. We utilize a clip function $\mathcal{C}$ to limit the original probability factor π into (0, 1), and then apply the normalization operation to each factor:

$$\pi_j = \frac{e^{\pi_j}}{\sum_{k=1}^K e^{\pi_k}} \rightarrow \frac{\mathcal{C}(\pi_j)}{\sum_{k=1}^K \mathcal{C}(\pi_k)}, \quad j = 1, \dots, K. \quad (14)$$

3) THE ARITHMETIC CODING MODULE

encodes symbols into more compact binary bitstreams using the obtained frequency range pairs under normal circumstances. However, due to limitations in computational precision or deviation introduced by either PDE module or MLAC module, unexpected cases will arise when the f_{low} are greater than or equal to the f_{high} . In such cases, arithmetic coding cannot compress these symbols. To address this issue, we introduce a protecting mechanism in which such excepted symbols are concatenated with their positions to form an extra data frame. It will cost up to 6 bytes for one excepted pixel of 16-bit datasets, which also proves the necessity of a high-precision Sigmoid approximation method to get a good compression ratio. Given that our PDE model estimates the logistic probability distribution of each pixel, this paper selects the multi-symbol AC for entropy coding rather than binary AC to maintain the compression ratio. On the other hand, due to the raster-scan order for our autoregressive layer, rANS [38] is unsuitable, especially for the decoding process.

D. SIGMOID FUNCTION

Until now, we have discussed the structure of our proposed framework, including the PDE module, and the MLAC module. In this subsection, we will introduce how we design a hardware-friendly approximation method of Sigmoid function for our specialized encoder.

Most existing sigmoid approximation methods are primarily designed for activation functions in neural networks and may not be suitable for an arithmetic coding based on the logistic probability distribution:

- 1) The accuracy requirement of the Sigmoid function in arithmetic coding is higher than the activation functions used in neural networks. For instance, in the deep learning classification task, converting the Sigmoid function to the Hard-Sigmoid function has little impact on the final accuracy [43]. However, replacing the Sigmoid function in arithmetic coding with a simple Hard-Sigmoid function significantly degrades the compression performance.
- 2) The Sigmoid approximation function used in arithmetic coding needs to satisfy the mathematical property of being monotonically increasing. Otherwise, during the process of computing the frequency range pairs, there is an increased probability of encountering exceptional situations, i.e., $low \geq high$, resulting in a worse compression ratio.
- 3) The sigmoid function used in arithmetic coding requires a larger effective input range. If, like many existing methods used for activation functions, only an input range of $(-8, 8)$ is considered, it would also result in a degradation in performance.

To address these challenges, we introduce a high-precision method based on the Newton-Raphson (NR) method. Motivated by the previous work [41], which first introduces the NR method into the approximation of the Sigmoid function, dividing the process of calculating the Sigmoid function into two steps. As shown in (15), the NR method first estimates a rough result σ_0 , typically based on the Piecewise Linear (PWL) method. Subsequently, it combines the reciprocal of the Sigmoid function to fine-tune a more accurate estimation by applying a further approximate calculation of e^{-x} :

$$\begin{aligned}\hat{\sigma}(x) &= 2\sigma_0 - \sigma_0^2 \frac{1}{\sigma} \\ &= 2\sigma_0 - \sigma_0^2(1 + e^{-x}).\end{aligned}\quad (15)$$

However, the hand-crafted PWL method used in [41] for the initial estimation σ_0 may lead to compression performance degradation. Inspired by the work in [39], this paper employs an adaptive optimization algorithm to obtain a group of piecewise linear functions with higher precision than [41], the whole process of which is as follows:

- 1) Uniformly divide the initial range $(0, 8)$ into sub-segments with stride of 1.

- 2) Perform linear approximation and calculate mean error with the golden Sigmoid function for each sub-segment.
- 3) Find the sub-segment with the largest mean error and divide it evenly into two new sub-segments, unless the interval of this sub-segment is less than or equal to 0.25.
- 4) Repeat the operation of 2 ~ 3 until the total number of sub-segments is more than or equal to 16.

Fig. 2.(a) illustrates the final result of PWL segments, where individual segments are plotted by distinct colors. This adaptive algorithm generates three kinds of segment types: type 1 with an interval equal to 0.25, type 2 with an interval equal to 0.5, and type 3 with an interval equal to 1.

After performing the initial approximation of σ_0 , we apply further fine-tuning by estimating the reciprocal of the Sigmoid function. Following [41], we partition the input range of x into segments of size $\ln 2 / 2^L$ to simplify the computation of e^{-x} , where L is a positive integer. Thus the new expression of input x and e^{-x} can be represented as follows:

$$x = (n + \frac{i}{2^L} + \varepsilon) \cdot \ln 2, \quad (16)$$

$$e^{-x} = 2^{-n} \cdot 2^{-i/2^L} \cdot 2^{-\varepsilon}, \quad (17)$$

where n, i are integers and ε is a small number close to zero. To implement the computation of e^{-x} on the hardware platform, we can multiply the input x with the reciprocal of $\ln 2$ and then select the integer bits as the value of n , the middle 2^L bits as the value of i , and the remain bits as the value of ε . Thus, we can achieve the computation of 2^{-n} by shift operations, and obtain the result of $2^{-i/2^L}$ by lookup tables (LUT).

Moreover, as ε is a small number close to zero, we can approximate the computation of $f(\varepsilon) = 2^{-\varepsilon}$ with a linear function where the slope is the derivative of $f(x)$ and the intercept equal to $f(x)$ as x approaches zero:

$$\begin{aligned}2^{-\varepsilon} &\approx \lim_{x \rightarrow 0} [f'(x) \cdot \varepsilon + f(x)] \\ &= -\ln 2 \cdot \varepsilon + 1.\end{aligned}\quad (18)$$

Finally, the entire process of estimating a Sigmoid function, as shown in (19), can be further expressed as the following one:

$$\begin{aligned}\hat{\sigma}(x) &= 2\sigma_0 - \sigma_0^2 \frac{1}{\sigma} \\ &= 2\sigma_0 - \sigma_0^2(1 + 2^{-n} \cdot 2^{-i/2^L} \cdot (-\ln 2 \cdot \varepsilon + 1)).\end{aligned}\quad (19)$$

Fig. 2.(c) demonstrates the Sigmoid function obtained through our method. Compared to the standard Sigmoid function as a golden reference, the deviation between the two curves is tiny. The green curve is obtained by the absolute error times 2500, which also proves the high accuracy of our method. Overall, this proposed Sigmoid approximation method enables our HENCE framework to be deployed on a hardware platform, while maintaining better compression

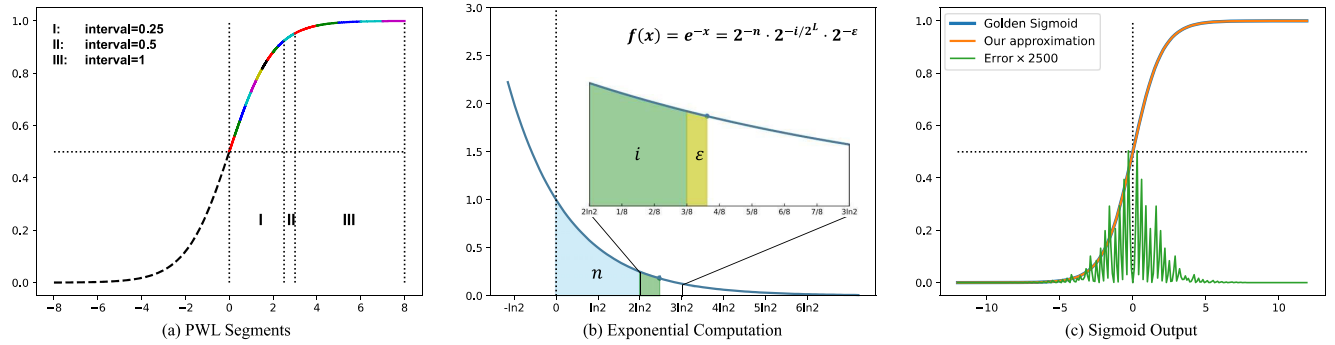


FIGURE 2. Our proposed approximation method to calculate the cumulative distribution function of the logistic distribution, i.e. Sigmoid function. We introduce a two-step Newton-Raphson (NR) method, which includes (a) initial piecewise linear (PWL) segments that are plotted with distinct colors, and (b) further fine-tuning where e^{-x} is calculated by multiplying three items of powers of 2. The final fitting result with the standard Sigmoid function is shown as (c).

performance than other Sigmoid approximation methods (more details will be introduced in Section IV-D).

E. DECODING

In autoregressive compression methods with Masked convolutional layers, decoding is usually more complex than encoding [18], [19], [22]. In our method, iterative masked convolutions should be applied to the already decoded symbols of the current s slice. Then, based on the auxiliary feature H_{s-1} from previous slices, MLAC module uses the outputted logistic distributions to recover symbols from bitstreams. Similarly, as in the existing literature, we use the CPU to calculate the CDF table and find the decoded symbol within the frequency range (*low*, *high*).

III. HARDWARE ARCHITECTURE

The entire hardware architecture of our HENCE is illustrated in Fig. 3. The input image X is first passed into the PDE neural network core, where we utilize a line buffer similar to SR-LVC [24] to cache input data, and use a sliding window based on shift registers to store variables involved in the current convolution iteration. As for the auxiliary feature H , we employ the same buffer mechanism but reduce the channel number to 16. Then, we apply a masked convolution module and a depthwise separable convolution module to extract intra-slice and inter-slice features, respectively. Due to the limited number of parameters used in our lightweight entropy model, all of the weights are stored by on-chip BRAM. Unlike SR-LVC, we design a more concise memory gate to fuse intra-slice and inter-slice feature information, i.e. the hard-tanh function module. Moreover, we use two pointwise convolution layers and one ReLU module for the mixed-logistic estimator to predict parameters of K logistic distributions, including μ , $\log(s)$, π . Considering the balance between compression efficiency and resource consumption as discussed in [12], we set $K = 3$ here, and more ablation experiments can be found in Section IV-D. In our HENCE architecture, once the PDE module successfully outputs parameters of the logistic distributions, the coding

module MLAC is responsible for encoding input pixels into binary bitstreams based on these distributions. First, We quantize the parameter mean μ and logarithmic scale $\log(s)$ using shift and truncation operations and restore the logarithmic scale $\log(s)$ to scale s through the corresponding LUT. On the other hand, the parameters also include the probability π of each logistic distribution. Ensuring the sum of all probabilities equals 1, we realize the normalization operation by dividing each probability by the sum of them. Furthermore, we instantiate three Sigmoid units to achieve parallel computation of multiple logistic CDFs. Combining each CDF result with the corresponding probability using multipliers and adders, the current pixel's symbol frequency pair (*low*, *high*) is finally outputted. Overall, we employ 16-bit fixed-point numbers during the computations of the PDE module and up to 24-bit fixed-point numbers in the MLAC module.

Fig. 3.(b) illustrates the internal structure of each sigmoid unit, which mainly includes two computation datapaths, one is for the initial PWL estimation σ_0 and the other is for fine-tuning through the NR method shown in (15). Specifically, in the process of computing σ_0 , we first determine the segment type based on the integer part of the input value. Then we output the select signals and the index signals to obtain the slope and intercept of the corresponding linear function. As for computing e^{-x} , we first use a multiplier to multiply the input by the value of $1/\ln 2$ stored in the register. Then we take the highest 8 bits of the result to represent the integer part, the lowest 20 bits to represent the remainder ϵ , which approximates zero, and the middle 4 bits as the middle term. After obtaining the values of σ_0 and e^{-x} , we use multipliers and adders to output the ultimate approximation result of the Sigmoid function.

The overall schedule is displayed in Fig. 4. Here we take four sequential pixels of the same slice as an example. Once the parameters involved in the current pixel are loaded to the sliding windows, the masked convolution on intra-frame feature extraction and the DSC module on inter-frame auxiliary feature extraction will be processed simultaneously.

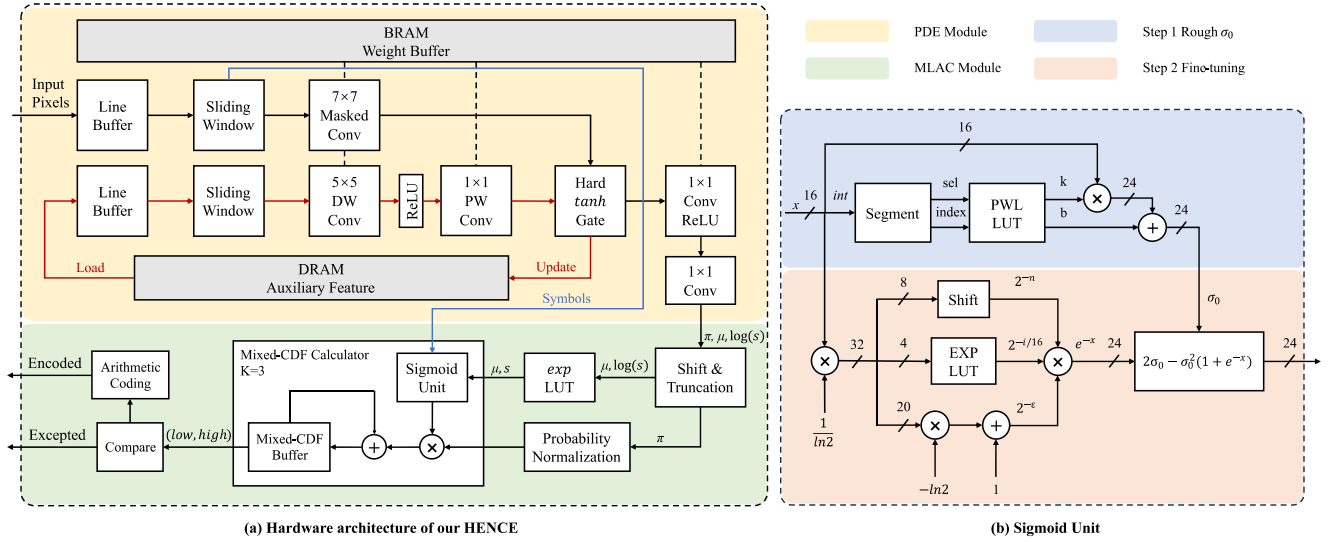


FIGURE 3. The details of (a) the overall hardware architecture and (b) the internal structure of each Sigmoid Unit. Within the PDE module, the recurrent auxiliary feature H in an inter-forward way is depicted by red arrows. The numbers above the arrows in (b) represent the bit widths of each fixed-point number in the hardware implementation.

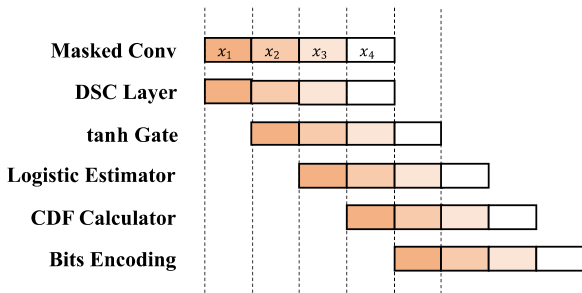


FIGURE 4. The pipelined schedule diagram of hardware implementation. Here x_i represents several sequential pixels of the same slice that need to be encoded.

After that, the $\tanh$ gate, the logistic estimator, the CDF calculator, and the coding module will all follow the common pipeline one after the other. Specifically, the auxiliary features will be updated back to the DRAM one by one as they are output by the $\tanh$ gate. Thus, the input symbols can be encoded sequentially in an efficient way. Once the entire image slice is finished, the auxiliary features are also updated, making it ready to encode the next slice.

For implementation, the Xilinx Vitis platform supports the design of hardware kernels using High-Level Synthesis [47] languages. In our study, we designed the PDE network kernel and the MLAC kernel respectively. By modifying the configuration files, we connected these two kernels to achieve end-to-end hardware acceleration for compression. In terms of data transmission, we utilized the OpenCL library to migrate raw images and output bits between host memory and hardware memory on FPGA.

IV. EXPERIMENTS

In this section, we will first introduce the datasets and the experimental settings. Then, we will compare our method with the state-of-the-art methods on a variety of datasets.

After that, the ablation study will be provided. Finally, we will present the implementation results and analyze the computational complexity.

A. DATASETS

1) MRNet

The MRNet dataset consists of knee MRI examinations conducted on 1199 patients, and 1370 knee examinations [48]. Each examination includes three scanning orientations: axial, coronal, and sagittal. Within each scanning orientation, a sequence of image slices is provided, ranging from 17 to 61 slices. The resolution of each image slice is 256×256 pixels, and they are stored with an 8-bit depth. The dataset has been divided into a training set and a validation set. The training set includes 1130 scans from 1088 patients, while the validation set comprises 120 scans from 113 patients. Table 2 summarizes the details of this dataset.

2) CHAOS

The CHAOS dataset [49] consists of abdominal CT and MRI images obtained from 40 patients. For our research, we select the CT images, the out-phase channel of T1DUAL MRI images, and the T2SPIR MRI images for lossless compression. Initially, these images are stored in 16-bit Digital Imaging and Communications in Medicine (DICOM) format. We organize the dataset by arranging the 40 patients in ascending order by index. Subsequently, we divide the images into a training set and a validation set. The training set includes 70% of the images (28 patients), while the validation set comprises the remaining 30% of the images (12 patients).

3) ACDC

The ACDC dataset [51] contains cardiac cine-MRI sequences obtained from 150 patients. We directly use the divided training and test sets. The raw input images are provided

TABLE 2. Summary of volumetric medical image datasets in our experiments.

	MRNET [48]	CHAOS [49]	SCD [50]	ACDC [51]
Type	MRI	CT+MRI	cine-MRI	cine-MRI
Bit Depth	8	16	16	16
Resolution	256*256	512*512	256*256	(154~428)*(154~512)
Slices per Volume	17-61	26-294	20-400	84-450
Training Volumes	1130	28	130	100
Testing Volumes	120	12	20	50

through the 16-bit nifti format, the resolution of which ranges from $(154 \sim 428) \times (154 \sim 512)$. Due to the higher dimensional characteristics of cine-MRI, following the method in DBVC [23], we split the data into ACDC-xyz type and ACDC-xyt type, where *xyz* refers to splitting the data along the spatial dimension *z*, and *xyt* refers to splitting the data along the time dimension *t*, obtaining multiple 3D cubes. Different from DBVC, which then compressed these cubes individually, we concatenate these cubes of the same patient to one big volume with slices ranging from 84 to 420.

4) SCD

The Sunnybrook Cardiac Data (SCD) [50], consists of 45 cine-MRI images from a mix of patients and pathologies, which are initially saved in the 16-bit DICOM format with the resolution of 256×256 . We extract the cine MRI images named CINELAX and CINESAX from all patients and sort them according to their numbers. In total, we obtain 150 3D volumes. Subsequently, we divide the volumes into a training set, consisting of the first 130 volumes, and a test set, consisting of the remaining 20 volumes.

B. EXPERIMENT SETTINGS

1) METRICS

We perform experiments to validate the effectiveness of our proposed framework with bits per pixel (bpp) for lossless compression. In this work, we follow the method in [24] to calculate the total bit cost.

2) BASELINE

We conduct a comparative analysis between our proposed approach and traditional lossless image compressors such as PNG [1], JPEG2000 [2], JP3D [3], FLIF [4] and JPEG-XL [5]. In addition, we select traditional video-based compression algorithms, including H.265/HEVC [7], FFV1 [8] and VVC [9], as baseline methods. For PNG compression, we utilize the Python Imaging Library (PIL). For JPEG2000 and JP3D, we use the OpenJPEG-2.3.1 from the OpenJPEG library with the default configuration. We use the default lossless configuration for FLIF-0.4 and JPEG-XL-0.10.2. FFmpeg-4.3 is used for compressing images with HEVC and FFV1. To achieve the optimal lossless compression ratio of HEVC, we configure the “x265-params” parameter into “lossless” and the “preset” parameter into “very slow”. Finally, we perform the default lossless and intra-frame configuration of VTM-23.3 on the MRNet datasets.

TABLE 3. Lossless compression results (in BPP) of our method and other state-of-the-art Cordex on the MRNet dataset.

	Axial	Coronal	Sagittal
PNG [1]	5.327	4.571	5.57
JP2K [2]	4.993	4.161	5.306
JP3D [3]	4.966	4.152	5.289
HEVC [7]	5.228	4.385	5.595
FFV1 [8]	4.865	4.049	5.203
FLIF [4]	4.861	4.009	5.160
JPEGXL [5]	4.727	3.880	5.107
VVC [9]	4.759	3.887	5.100
L3C [22]	5.189	4.342	5.598
ICEC [10]	4.642	3.841	4.974
aiWave [21]	4.545	3.804	4.829
SR-LVC [24]	4.474	3.707	4.862
Ours	4.488	3.689	4.852

Regarding learning-based lossless image compression, we compare our method with L3C [22], ICEC [10], aiWave [21], DBVC [23], and SR-LVC [24]. However, it is worth noting that L3C is unable to compress datasets with a bit depth greater than 8 bits.

3) TRAINING SETTING

We adopt a unified network architecture for all experiments. For feature extraction of intra-forward route *X*, we apply a single masked convolution layer with a kernel size of 7 and kernel number of 16. Moreover, we set the channel number of the auxiliary features *H* to 16, with width and height both equal to the input images. To further reduce the computation complexity, we set the kernel size of depth separable convolution to 5.

Typically, the training method for recurrent neural networks involves the Back Propagation Through Time (BPTT) algorithm [52]. However, when dealing with long-time sequences, the issue of vanishing gradients [53] arises, corresponding to the large number of slices per volume in our datasets. Additionally, in datasets with a large bit depth, it is challenging to achieve a great loss reduction during the training process. To address these issues, following [24], we set the update stride as 5, set the scaling factor *L* to 1 for 8-bit depth datasets, and set *L* to 8 for 16-bit depth datasets.

To train our model on these datasets, we initialize the learning rate as 5×10^{-4} , set the total number of 1000 epochs, and decay the learning rate as 80% after every 50 epochs. We also use the Adam optimizer in our

TABLE 4. Lossless compression results (in BPP) of our method and other state-of-the-art Cordecos on the CHAOS Dataset.

	CT	MRI T1	MRI T2
PNG [1]	7.472	5.341	5.38
JP2K [2]	5.227	3.753	3.758
JP3D [3]	5.287	3.879	3.881
HEVC [7]	5.245	4.056	4.07
FFV1 [8]	5.303	3.714	3.746
FLIF [4]	5.244	3.546	3.593
JPEGXL [5]	4.993	3.412	3.520
aiWave [21]	5.077*		
SR-LVC [24]	4.293	3.312	3.471
Ours	4.354	3.270	3.406

* Reference only for different data pre-processing method.

experiment. Unlike [10], [21], [23], we do not apply any data augmentation method.

4) PLATFORM

During the training and inference phases, we use the NVIDIA RTX 3090 GPU and the Intel(R) Xeon(R) Gold 6128 CPU. Our code is implemented by Python 3.11 on PyTorch 2.0 framework. Finally, we utilize Vitis-2022.1 [47] for hardware implementation to deploy our framework on the Xilinx Alveo U250.

C. LOSSLESS COMPRESSION RESULTS

1) MRNet DATASETS

From Table 3, it can be seen that our method achieves a better compression ratio (i.e. lower bpp) than all the traditional compression methods. Compared with learning-based methods, our framework outperforms the L3C and ICEC methods in all sub-datasets. Particularly, in the Coronal dataset, our proposed method achieves the best compression ratio.

In detail, in the Axial dataset, our method achieves a 5.05% improvement in compression ratio compared to the best traditional compression algorithm JPEG-XL. As for learning-based methods, our method also has 3.32% and 1.25% bpp reductions than ICEC and aiWave respectively. Considering the Coronal sub-dataset, our method achieves an 4.93% improvement in comparison with JPEG-XL and saves 0.49% bpp over the SR-LVC method. Lastly, in the Sagittal sub-dataset, VVC is the optimal method among all the traditional codecs, and our method achieves a 4.87% improvement over VVC.

It is noteworthy that although our method does not achieve the best compression performance on all datasets, considering the computational complexity and resource utilization, our proposed method has a more lightweight model and a faster compression speed than other network-based methods. More discussions will be presented in Section IV-E.

2) CHAOS DATASETS

In this dataset, we compare our method with traditional compression methods, as well as aiWave and SR-LVC,

TABLE 5. Lossless compression results (in BPP) of our method and other state-of-the-art Cordecos on the ACDC and SCD datasets.

	ACDC-xyt	ACDC-xyz	SCD
PNG [1]	5.302	5.302	5.395
JP2K [2]	3.737	3.737	3.970
JP3D [3]	5.124	5.129	4.081
HEVC [7]	2.773	3.655	3.317
FFV1 [8]	3.587	3.598	3.800
FLIF [4]	3.578	3.578	3.763
JPEGXL [5]	3.312	3.312	3.587
aiWave [21]		3.921*	
DBVC [23]		3.521*	
SR-LVC [24]	2.477	3.348	2.757
Ours	2.523	3.229	2.826

* Image and video coding methods utilizing 3-D convolutions are similar to the xyz format.

as shown in Table 4. It should be noted that aiWave only conducted experiments on the CHAOS-CT dataset, and its data pre-processing method differs from ours. In detail, aiWave crops the 3D volumes into $64 \times 64 \times 64$ cubes and discards those with more than 50% of zero-value pixels. Therefore, the experimental result of aiWave is for reference only.

In the CT sub-dataset, the best traditional method is JPEG-XL, and our method achieves a 12.79% improvement in compression ratio compared to it. In the T1 MRI and T2 MRI datasets, the best traditional compression method is also JPEG-XL, which has demonstrated its outstanding compression performance in MRNet dataset. Nevertheless, our method still outperforms it by achieving bpp reductions of 4.16% and 3.25% in these two sub-datasets, respectively. When compared to SR-LVC, our method achieves ratio improvements of 1.27% and 1.87% for compressing T1 MRI and T2 MRI datasets.

3) ACDC DATASET

The compression results, as shown in Table 5, indicate that in the ACDC-xyt sub-dataset, the traditional method with the best compression performance is HEVC. It outperforms PNG with a 47.7% improvement in compression ratio and a 45.6% improvement over JP3D. Our method, compared to HEVC, still achieves a 9% bpp reduction. In the ACDC-xyz sub-dataset, the best-performing traditional method is JPEG-XL, with a 7.95% improvement over FFV1. Our method still exhibits a 2.49% bit-saving over JPEG-XL.

Within each volume in the ACDC dataset, there is a high similarity between slices, especially in the ACDC-xyt subset where frames are ordered in time series. Therefore, traditional video-based compression methods are more effective in reducing data redundancy. In our experiments, we set HEVC's runtime parameters to "very slow", which achieves the optimal compression ratios among traditional methods. Our method utilizes memory gates to remember inter-frame information and employs a channel-rich auxiliary feature to preserve those prior features, encompassing expressive

TABLE 6. Summary of place and route results for MRNet-Coronal dataset on Xilinx Alveo U250 platform.

		Bram18K	DSP	FF	LUT	Clock (MHz)	Framerate (fps)	Power (W)
Network	SR-LVC [24]	503	1174	74031	123182	243	120	4.773
	Our PDE	342	1072	97332	84774	250	200	4.572
Coding	Our MLAC	13	74	27365	27463	300	320	1.37
Total	PDE+MLAC	355	1146	124697	112237	250	200	5.942

TABLE 7. Average CPU encoding time per image (in Seconds) on the MRNet-Coronal dataset and the number of training parameters.

Method	BPP	Encoding (s)	Decoding (s)	#Params
PNG [1]	4.571	0.0158	0.0002	-
JP2K [2]	4.161	0.0426	0.0212	-
JP3D [3]	4.152	0.0318	0.0167	-
HEVC [7]	4.385	0.1302	0.0067	-
FFV1 [8]	4.049	0.0042	0.0052	-
FLIF [4]	4.009	0.0828	0.0314	-
JPEGXL [5]	3.880	0.0768	0.0375	-
VVC [9]	3.887	5.2539	0.0118	-
L3C [22]	4.342	0.5360	-	5M
ICEC [10]	3.841	1.1550	-	17M
aiWave [21]	3.804	986.9660*	982.0320*	699.5MB
SR-LVC [24]	3.707	0.0860	0.3159	4.8K
Ours	3.689	0.0460	0.2010	1.5K

* Reference only for time per slice of FAFB dataset.

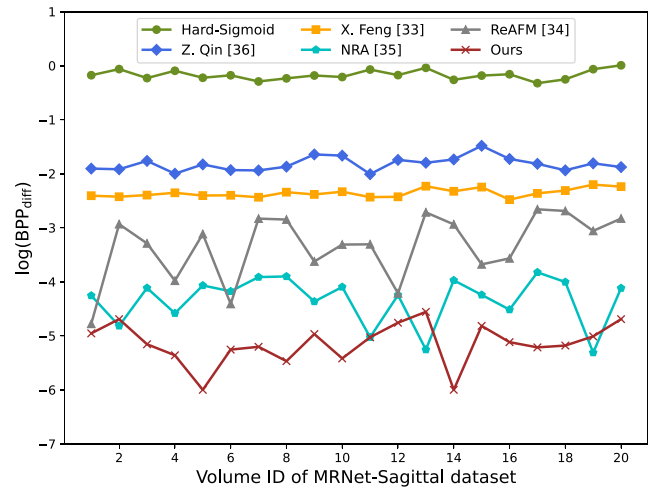
contextual content. Thus, our process is more concise and efficient when compared to DBVC [23], which uses additional network modules for inter-frame motion estimation and compensation.

4) SCD DATASET

The SCD dataset is also a cine-MRI dataset. Similar to the pre-processing method of ACDC-xyt sub-dataset, we divide the raw data into multiple 3D cubes in the temporal order and then concatenate them to a large volume. The final compression results are shown in Table 5, among traditional methods, HEVC still achieves the best compression performance. It brings an 18.7% improvement in compression ratio over JP3D and a 12.7% improvement over FFV1. Moreover, our method surpasses all traditional image compression methods. Compared to JP2K and JP3D, our method outperforms them with 28.8% and 30.75% bpp reductions, respectively. When compared to video compression methods HEVC and FFV1, our method still achieves compression ratio improvements of 14.80% and 25.62%.

D. HARDWARE IMPLEMENTATION

Table 6 presents the resource utilization results of our HENCE framework on Alveo U250 after place and route, for MRNet-Coronal datasets with the image resolution of 256×256 . We compare our approach with the SR-LVC method on the entropy model module. From the table, it can be observed that our method occupies fewer resources and has lower power consumption while achieving nearly twice the inference speed compared to SR-LVC. Specifically, our

**FIGURE 5.** Logarithm of the lossless compression differences between approximation methods with the standard Sigmoid on the MRNet-Sagittal dataset. First, we alternate six approximation Sigmoid functions and the standard Sigmoid function in our MLAC module, obtaining each lossless compression ratio (in BPP). Then we calculate the ratio differences between each approximation Sigmoid with the standard one. Last, we do a logarithm of these ratio differences.

PDE module consumes 342 BRAMs, 1072 DSPs, 97332 FFs, and 84774 LUTs. The synthesized clock frequency reaches 250 MHz, and the frame rate of inference for one slice reaches 200 fps. Furthermore, for the coding, the MLAC module proposed in this paper employs 13 BRAMs, 74 DSPs, 27365 FFs, and 27463 LUTs, whose encoding speed can achieve 320 fps. Finally, by integrating the PDE module and MLAC module into our complete HENCE system, both the clock frequency and frame rate will be reduced to the lower value between the two modules, exemplifying the ‘weakest link’ effect where the performance is limited by the least capable component, with a total power consumption lower than 6 W.

E. COMPUTATIONAL COMPLEXITY

To analyze the computational complexity of our method, we employ CPU coding time as the evaluation criterion for ease of comparison with other methods. Using the MRNet-Coronal dataset as an example, we record the average time each method takes to compress a single slice. For learning-based methods, this encoding time includes both the network inference time and the entropy coding time. All the experiments of traditional methods and ours are performed on the machine with an Intel(R) Xeon(R) Gold 6128 CPU. Moreover, we apply OpenMP technology to accelerate our framework by 16 threads. The detailed results are shown in

Table 7. Notably, the encoding time of aiWave on the table is for another FAFB dataset, which means the result of aiWave is only for reference.

Among deep learning-based medical compression approaches, our framework achieves the fastest coding speed, even comparable to traditional compression methods. In addition, our method requires the fewest parameters, only 1.5K, which is less than one-third of SR-LVC. When compared to ICEC and aiWave, our method saves a significant amount of computational resources, further reducing model complexity, which plays a decisive role in facilitating the deployment of our HENCE framework on FPGA platforms.

F. ABLATION STUDY AND ANALYSIS

1) COMPARISON WITH SIGMOID FUNCTIONS

In Section II, we have introduced our high-precision Sigmoid approximation method in the MLAC module. We also highlight the significant impact of accurate CDF computations on the final compression performance during practical coding. In this subsection, we conduct further experiments by comparing various existing approximation methods of the Sigmoid function. We first train the PDE module on the MRNet dataset to obtain the optimal model parameters. Then, by alternating the Sigmoid function within the MLAC module, we obtain compression ratios (BPP) for the standard Sigmoid function and the approximation Sigmoid functions. Then we calculate the absolute difference between the compression ratios of these fitted functions and the standard function, followed by taking the logarithm of this absolute difference.

Fig. 5 presents the results for the first 20 volumes in the MRNet-Sagittal dataset, where the horizontal axis represents the number of volume IDs and the vertical axis represents the corresponding logarithm of BPP differences. It can be observed that using the hard-sigmoid function to compute the CDF deviates significantly from the standard sigmoid function in terms of compression performance. Our method demonstrates better compression performance than other Sigmoid approximation methods.

Additionally, from Fig. 5, we can see that the three curves at the bottom, corresponding to methods [40], [41], and ours, exhibit smaller errors with the standard Sigmoid function. These three methods all utilize the Newton-Raphson approximation method, which plays a decisive role in the fitting accuracy. However, in the ReAFM method [40], shift operations similar to [42] are employed for numerical approximation, which may lead to a decrease in compression performance. For example, the reciprocal of $\ln 2$ is calculated by $\frac{1}{\ln 2} \approx 1 + (1 \gg 1) - (1 \gg 4)$. In contrast, both NRA [41] and our method directly use registers to store high-precision values for such frequently used constant variables. Additionally, the PWL linear estimation strategy in the NRA method is relatively crude, while in our method,

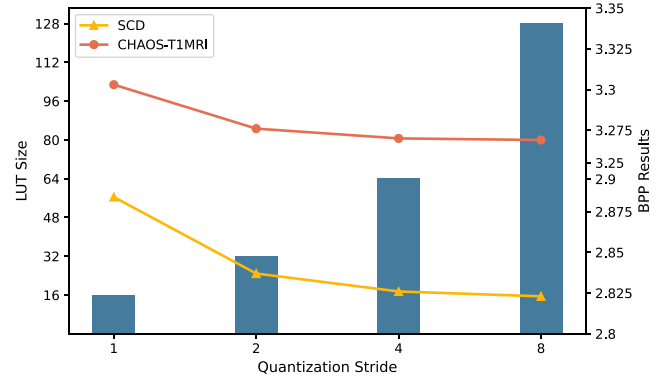


FIGURE 6. Lossless compression results (in BPP) and estimated hardware LUT size with different quantization stride of the logarithmic scale on the SCD and CHAOS-MRI T1 datasets.

TABLE 8. Lossless compression results (in BPP) of our method with the different number K of mixed distributions on the CHAOS-MRI T2 and MRNet-Axial datasets.

K	MRNet-Axial	CHAOS_T2
1	4.524	3.429
2	4.517	3.419
3	4.489	3.406
4	4.487	3.422
5	4.487	3.427

the PWL step employs an adaptive optimization strategy, resulting in higher precision.

2) ANALYSIS OF LOGSCALE QUANTIZATION STRIDE

In Section II, we have mentioned that deep learning-based image compression methods typically predict the mean μ and the log scale $\log(s)$ of logistic distributions in the PDE network module. To obtain the value of the original scale, we introduce a hardware-friendly two-step approach, using a quantization operation and a lookup table, as shown in (6). To investigate the impact of the quantization stride E on compression performance, we conduct more experiments on the SCD and CHAOS-T1MRI datasets, by varying the quantization stride of $\log(s)$ and then recording the corresponding compression ratios. Results are plotted in Fig. 6, the bar chart represents the approximate size of LUTs for each quantization stride setting, while the line charts represent the compression results of those two datasets. Here we estimate the LUT size simply by $16 \times E$ because the value of $\log(s)$ output by the PDE module is almost within the range of $(-16, 0)$.

From Fig. 6, it can be observed that with the increase of the quantization stride K , the corresponding compression ratio decreases, but the required size of LUTs increases rapidly. Additionally, the greatest improvement in compression ratios is achieved when K increases from 1 to 2. However, when it increases from 4 to 8, the bpp results on both datasets show minimal declines, indicating limited performance improvement. Therefore, considering the balance between

TABLE 9. Lossless compression results (in BPP) of our method with different gate module on the CHAOS-MRI T1 and CHAOS-MRI T2 datasets.

	MRI T1	MRI T2
w/o auxiliary	3.337	3.510
w/o gate	3.315	3.444
ours	3.270	3.406

TABLE 10. Technical requirement of different learning methods.

	aiWave	L3C	ICEC	Ours
Multi-Resolution	×	✓	×	✓
Inter Slice	✓	×	✓	✓
Hardware Cost	High	High	High	Low
Ratio	High	Low	Medium	High
Latency	High	Medium	Medium	Low
Parameters	Heavy	Medium	Medium	Light

resource and performance, we choose the quantization stride E of 4.

3) ANALYSIS OF LOGISTIC MIXTURE NUMBER

To investigate the impact of the number of mixed logistic distributions on compression performance, we conduct further experiments on the MRNet-Axial dataset and the CHAOS-MRI T2 dataset. Specifically, we enumerate the factor K from 1 to 5 on both datasets, which represents the number of mixed distributions for one symbol, and retrain the entropy model PDE.

Table 8 shows the final results. On the MRNet-Axial dataset, there is a continuous decrease in compression ratios (BPP) with an increasing number of distributions (K). However, for the CHAOS-MRI T2 dataset, as K increases from 3 to 5, compression performance becomes even worse. This fact shows that increasing the number of mixed distributions does not necessarily lead to improved compression performance, especially when the dataset size is limited, as in the case of the CHAOS-MRI dataset. Moreover, considering the hardware implementation of our proposed framework, increasing the number of distributions (K) will result in the mixed-logistic estimator in the PDE module and the mixed-CDF calculator in the MLAC module consuming more logical units, longer computation delays, and more complex temporal conditions. Therefore, to balance resource and performance, we set K to 3 in our method.

4) ANALYSIS OF GATE MODULE

To investigate the impact of the $\tanh$ gate module on compression performance, we conduct additional ablation experiments on the CHAOS-MRI T1 and T2 datasets. As shown in Table 9, we first remove the auxiliary features during the gate module fusion process, which is indicated as “w/o auxiliary” in the table. Next, we remove the non-linear $\tanh$ function from the gate module and replace it with a sum, which is indicated as “w/o gate” in the table. The results demonstrate that both the auxiliary features and the $\tanh$

module used in our method contribute to the improvement of lossless compression performance.

5) ANALYSIS OF TECHNOLOGICAL REQUIREMENTS

We compare our HENCE model with other learning-based methods to meet the compatibility of technological requirements in Table 10. The columns represent whether flexible multi-resolution input is supported, whether the inter-slice redundancy is considered, whether hardware acceleration is conducted, and the compression performance metrics such as ratio, latency, and number of parameters.

V. CONCLUSION

This paper proposes a fully hardware end-to-end neural entropy encoding framework HENCE for lossless volumetric medical image compression. We first introduce a context-based entropy model as a simple baseline, which exploits information from both inter-slice and intra-slice to reduce data redundancy. Then, we design a coding module MLAC, which includes a high-precision Sigmoid approximation method specialized for arithmetic coding. After that, we propose a pipelined architecture to implement the whole framework on FPGA. The experimental results demonstrate that our approach achieves better compression ratios than the next-generation image codec JPEG-XL and greatly improves the encoding speed of learning-based compression methods.

ACKNOWLEDGMENT

(Jietao Chen and Qianhao Chen contributed equally to this work.)

REFERENCES

- [1] G. Roelofs, *PNG: the Definitive Guide*. Sebastopol, CA, USA: O'Reilly & Associates, 1999.
- [2] D. S. Taubman and M. W. Marcellin, “JPEG2000: Standard for interactive imaging,” *Proc. IEEE*, vol. 90, no. 8, pp. 1336–1357, Aug. 2002.
- [3] P. Schelkens, A. Munteanu, A. Tzannes, and C. Brislawn, “JPEG2000 Part 10—Volumetric data encoding,” in *Proc. IEEE Int. Symp. Circuits Syst.*, Apr. 2006, pp. 3874–3877.
- [4] J. Sneyers and P. Wuille, “FLIF: Free lossless image format based on MANIAC compression,” in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2016, pp. 66–70.
- [5] J. Alakuijala, R. Van Asseldonk, S. Boukourt, M. Bruse, I.-M. Comşa, M. Firsching, T. Fischbacher, E. Kliuchnikov, S. Gomez, and R. Obryk, “JPEG XL next-generation image compression architecture and coding tools,” *Proc. SPIE*, vol. 11137, pp. 112–124, May 2019.
- [6] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, “Overview of the H. 264/AVC video coding standard,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 7, pp. 560–576, Jul. 2003.
- [7] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, “Overview of the high efficiency video coding (HEVC) standard,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 22, no. 12, pp. 1649–1668, Dec. 2012.
- [8] M. Niedermayer. (2013). *FFV1 Video Codec Specification*. [Online]. Available: <https://www.ffmpeg.org/michael/ffv1.html>
- [9] B. Bross, Y.-K. Wang, Y. Ye, S. Liu, J. Chen, G. J. Sullivan, and J.-R. Ohm, “Overview of the versatile video coding (VVC) standard and its applications,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 10, pp. 3736–3764, Oct. 2021.
- [10] Z. Chen, S. Gu, G. Lu, and D. Xu, “Exploiting intra-slice and inter-slice redundancy for learning-based lossless volumetric image compression,” *IEEE Trans. Image Process.*, vol. 31, pp. 1697–1707, 2022.

- [11] D. Rossinelli, G. Fourester, F. Schmidt, B. Busse, and V. Kurtcuoglu, "High-throughput lossy-to-lossless 3D image compression," *IEEE Trans. Med. Imag.*, vol. 40, no. 2, pp. 607–620, Feb. 2021.
- [12] N. Kang, S. Qiu, S. Zhang, Z. Li, and S. Xia, "PILC: Practical image lossless compression with an end-to-end GPU oriented neural framework," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 3729–3738.
- [13] S. Gao, Y. Zhang, D. Liu, and Z. Xiong, "Volumetric end-to-end optimized compression for brain images," in *Proc. IEEE Int. Conf. Vis. Commun. Image Process. (VCIP)*, Dec. 2020, pp. 503–506.
- [14] J. Ballé, D. Minnen, S. Singh, S. Jin Hwang, and N. Johnston, "Variational image compression with a scale hyperprior," 2018, *arXiv:1802.01436*.
- [15] H. Ma, D. Liu, N. Yan, H. Li, and F. Wu, "End-to-end optimized versatile image compression with wavelet-like transform," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1247–1263, Mar. 2022.
- [16] F. Kamisli, "Learned lossless image compression through interpolation with low complexity," *IEEE Trans. Circuits Syst. Video Technol.*, p. 1, 2023.
- [17] H. Ma, D. Liu, R. Xiong, and F. Wu, "IWAVE: CNN-based wavelet-like transform for image compression," *IEEE Trans. Multimedia*, vol. 22, no. 7, pp. 1667–1679, Jul. 2020.
- [18] D. Minnen, J. Ballé, and G. D. Toderici, "Joint autoregressive and hierarchical priors for learned image compression," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [19] A. Van Den Oord, N. Kalchbrenner, and K. Kavukcuoglu, "Pixel recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1747–1756.
- [20] T. Salimans, A. Karpathy, X. Chen, and D. P. Kingma, "PixelCNN++: Improving the PixelCNN with discretized logit mixture likelihood and other modifications," 2017, *arXiv:1701.05517*.
- [21] D. Xue, H. Ma, L. Li, D. Liu, and Z. Xiong, "AiWave: Volumetric image compression with 3-D trained affine wavelet-like transform," *IEEE Trans. Med. Imag.*, vol. 42, no. 3, pp. 606–618, Mar. 2023.
- [22] F. Mentzer, E. Agustsson, M. Tschanen, R. Timofte, and L. Van Gool, "Practical full resolution learned lossless image compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10621–10630.
- [23] D. Xue, H. Ma, L. Li, D. Liu, Z. Xiong, and H. Li, "DBVC: An end-to-end 3-D deep biomedical video coding framework," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 4, pp. 2922–2933, Apr. 2024.
- [24] Q. Chen and J. Chen, "Streaming lossless volumetric compression of medical images using gated recurrent convolutional neural network," 2023, *arXiv:2311.16200*.
- [25] J. Townsend, T. Bird, and D. Barber, "Practical lossless compression with latent variables using bits back coding," 2019, *arXiv:1901.04866*.
- [26] F. H. Kingma, P. Abbeel, and J. Ho, "Bit-swap: Recursive bits-back coding for lossless compression with hierarchical latent variables," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2019, pp. 3408–3417.
- [27] J. Townsend, T. Bird, J. Kunze, and D. Barber, "HiLLoC: Lossless image compression with hierarchical latent variable models," 2019, *arXiv:1912.09953*.
- [28] E. Hoogeboom, J. W. T. Peters, R. Van Den Berg, and M. Welling, "Integer discrete flows and lossless compression," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, May 2019, pp. 12134–12144.
- [29] S. Zhang, N. Kang, T. Ryder, and Z. Li, "IFlow: Numerically invertible flows for efficient lossless compression via a uniform coder," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2021, pp. 5822–5833.
- [30] H. Jun, R. Child, M. Chen, J. Schulman, A. Ramesh, A. Radford, and I. Sutskever, "Distribution augmentation for generative modeling," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5006–5019.
- [31] B. J. Frey and G. E. Hinton, "Efficient stochastic source coding and an application to a Bayesian network source model," *Comput. J.*, vol. 40, no. 2, pp. 157–165, Jan. 1997.
- [32] R. van den Berg, A. A. Gritsenko, M. Dehghani, C. K. Sønderby, and T. Salimans, "IDF++: Analyzing and improving integer discrete flows for lossless compression," 2020, *arXiv:2006.12459*.
- [33] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
- [34] C. Jia, X. Hang, S. Wang, Y. Wu, S. Ma, and W. Gao, "FPX-NIC: An FPGA-accelerated 4K ultra-high-definition neural video coding system," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 9, pp. 6385–6399, Sep. 2022.
- [35] K. Sun, M. Koch, Z. Wang, S. Jovanovic, H. Rabah, and S. Simon, "An FPGA-based residual recurrent neural network for real-time video super-resolution," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 4, pp. 1739–1750, Apr. 2022.
- [36] X. Wu, Y. Ma, M. Wang, and Z. Wang, "A flexible and efficient FPGA accelerator for various large-scale and lightweight CNNs," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 3, pp. 1185–1198, Mar. 2022.
- [37] P. G. Howard and J. S. Vitter, "Arithmetic coding for data compression," *Proc. IEEE*, vol. 82, no. 6, pp. 857–865, Jun. 1994.
- [38] J. Duda, "Asymmetric numeral systems: Entropy coding combining speed of Huffman coding with compression rate of arithmetic coding," 2013, *arXiv:1311.2540*.
- [39] X. Feng, Y. Li, Y. Qian, J. Gao, W. Cao, and L. Wang, "A high-precision flexible symmetry-aware architecture for element-wise activation functions," in *Proc. Int. Conf. Field-Program. Technol. (ICFPT)*, Dec. 2021, pp. 1–4.
- [40] X. Wu, S. Liang, M. Wang, and Z. Wang, "ReAFM: A reconfigurable nonlinear activation function module for neural networks," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, p. 1, 2023.
- [41] Z. Pan, Z. Gu, X. Jiang, G. Zhu, and D. Ma, "A modular approximation methodology for efficient fixed-point hardware implementation of the sigmoid function," *IEEE Trans. Ind. Electron.*, vol. 69, no. 10, pp. 10694–10703, Oct. 2022.
- [42] Z. Qin, Y. Qiu, H. Sun, Z. Lu, Z. Wang, Q. Shen, and H. Pan, "A novel approximation methodology and its efficient VLSI implementation for the sigmoid function," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 12, pp. 3422–3426, Dec. 2020.
- [43] M. Courbariaux, Y. Bengio, and J. David, "BinaryConnect: Training deep neural networks with binary weights during propagations," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, Dec. 2015, pp. 3123–3131.
- [44] Z. Zhao, R. Cao, K.-F. Un, W.-H. Yu, P.-I. Mak, and R. P. Martins, "An FPGA-based transformer accelerator using output block stationary dataflow for object recognition applications," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 1, pp. 281–285, Jan. 2023.
- [45] L. Xuan, K.-F. Un, C.-S. Lam, and R. P. Martins, "An FPGA-based energy-efficient reconfigurable depthwise separable convolution accelerator for image recognition," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 10, pp. 4003–4007, Oct. 2022.
- [46] R. Collobert, "Large scale machine learning," Université de Paris VI, Paris, France, Tech. Rep., 2004.
- [47] *Vitis High-Level Synthesis User Guide*, 2022.
- [48] N. Bien et al., "Deep-learning-assisted diagnosis for knee magnetic resonance imaging: Development and retrospective validation of MRNet," *PLOS Med.*, vol. 15, no. 11, Nov. 2018, Art. no. e1002699.
- [49] A. E. Kavur et al., "CHAOS challenge-combined (CT-MR) healthy abdominal organ segmentation," *Med. Image Anal.*, vol. 69, Apr. 2021, Art. no. 101950.
- [50] P. Radau, Y. Lu, K. Connelly, G. Paul, A. J. Dick, and G. A. Wright, "Evaluation framework for algorithms segmenting short axis cardiac MRI," *MIDAS J.*, Jul. 2009.
- [51] O. Bernard, O. Bernard, A. Lalande, C. Zotti, F. Cervenansky, X. Yang, and P.-A. Heng, "Deep learning techniques for automatic MRI cardiac multi-structures segmentation and diagnosis: Is the problem solved?" *IEEE Trans. Med. Imag.*, vol. 37, no. 11, pp. 2514–2525, Nov. 2018.
- [52] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proc. IEEE*, vol. 78, no. 10, pp. 1550–1560, Oct. 1990.
- [53] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.



JIETAO CHEN received the B.S. degree in measurement technology and instruments from Zhejiang University, Hangzhou, China, in 2020, where he is currently pursuing the Ph.D. degree with the College of Biomedical Engineering & Instrument Science. His research interests include image compression and computer vision.



QIANHAO CHEN received the B.S. degree in measurement technology and instruments and the Ph.D. degree in instrument science and technology from Zhejiang University, Hangzhou, China, in 2018 and 2024, respectively. His current major research interests include real-time AI and medical image processing.



WEI LUO received the B.S. degree in information and computing science and the Ph.D. degree in instrument science and technology from Zhejiang University, Hangzhou, China, in 2014 and 2020, respectively. From 2020 to 2022, he held a postdoctoral research position with The Chinese University of Hong Kong. He is currently an Assistant Professor with the College of Biomedical Engineering & Instrument Science, Zhejiang University. His major research interest includes biomedical applications with artificial intelligence.



HUAN ZHANG received the B.S. degree in measurement technology and instruments and the Ph.D. degree in instrument science and technology from Zhejiang University, Hangzhou, China, in 2013 and 2019, respectively. She is currently a Professor with the School of Communication Engineering, Hangzhou Dianzi University. Her current major research interests include database acceleration and embedded computer systems.



WEIJIE CHEN received the Ph.D. degree in instrument science and technology from Zhejiang University, Hangzhou, China, in 2020. From 2020 to 2022, he held a postdoctoral research position with Zhejiang University. He is currently an Assistant Professor with the College of Biomedical Engineering & Instrument Science, Zhejiang University. His current research interests include medical image processing and compression.



FENG YU (Member, IEEE) is currently a Professor and a Ph.D. Advisor with the College of Biomedical Engineering & Instrument Science, Zhejiang University.

...